EXP: 9 ARCHITECTURE DIAGRAM

AIM: To design and develop an ARCHITECTURE DIAGRAM for the system "PRECISION DISASTER RECOVERY (PDR-SYNC)" representing the overall structure, modules, and communication flow of the system.

THEORY: An ARCHITECTURE DIAGRAM represents the high-level structure of a software system. It shows system components, layers, communication flow, and interactions between modules.

The proposed system follows a layered architecture with:

- UI Layer
- Business Logic Layer
- Data Layer

The system also supports:

- Offline-first synchronization
- Mesh networking
- Data normalization
- Confidence scoring

COMPONENTS OF ARCHITECTURE DIAGRAM

UI LAYER: Handles user interaction and dashboard display

BUSINESS LOGIC LAYER: Processes normalization, confidence scoring, and synchronization

DATA LAYER: Stores resources, queue data, and mesh node details

MESH NETWORKING MODULE: Supports peer-to-peer communication between devices

COMMAND QUEUE: Handles offline synchronization and replay operations

NOTATIONS USED

NOTATION	DESCRIPTION
Rectangle	Module / Component
Arrow	Data Flow

Layered Block	System Layer
Connector Line	Communication

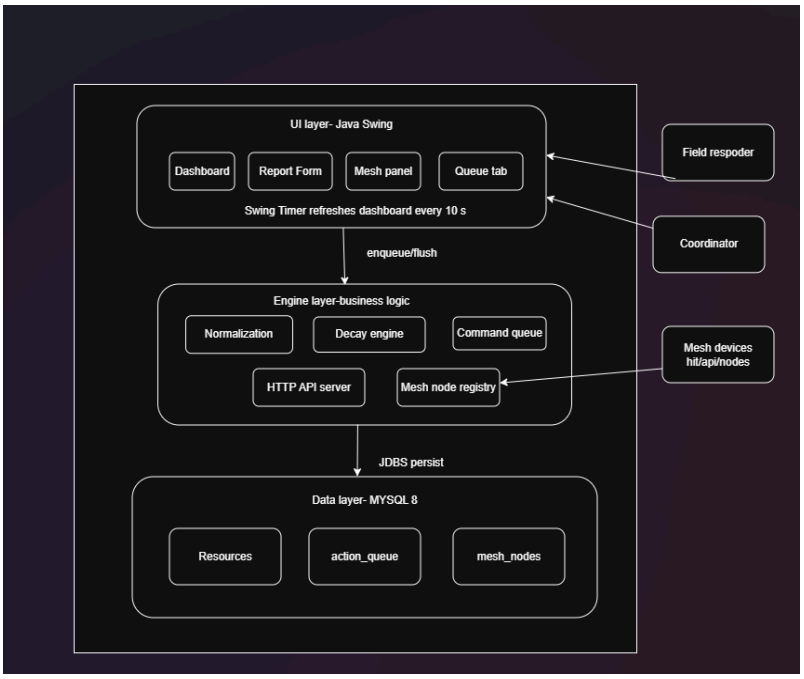
PROCEDURE:

1. Identify major system layers
2. Define components inside each layer
3. Establish communication between modules
4. Identify data storage components
5. Add synchronization and networking modules
6. Represent flow of data between layers
7. Validate architecture with system requirements

STEPS TO CREATE ARCHITECTURE DIAGRAM IN AZURE

1. Login to Azure Portal
2. Open Azure DevOps Project
3. Navigate to Repos or Wiki
4. Use draw.io or Lucidchart for architecture design
5. Create layered architecture components
6. Connect modules using arrows
7. Export architecture diagram as image
8. Upload diagram into Azure DevOps

ARCHITECTURE DIAGRAM



EXPLANATION OF ARCHITECTURE DIAGRAM

The architecture of PDR-SYNC consists of three major layers.

UI LAYER – JAVA SWING: The UI layer provides interaction between users and the system.

Modules include:

- Dashboard
- Report Form
- Mesh Panel
- Queue Tab

Field responders and coordinators interact through this layer.

The Swing timer refreshes the dashboard periodically.

ENGINE LAYER – BUSINESS LOGIC

This layer contains core system functionality.

Modules include:

- Normalization Engine
- Decay Engine
- Command Queue
- HTTP API Server
- Mesh Node Registry

Functions performed:

- Unit normalization
- Confidence score calculation
- Offline queue handling
- Mesh device management

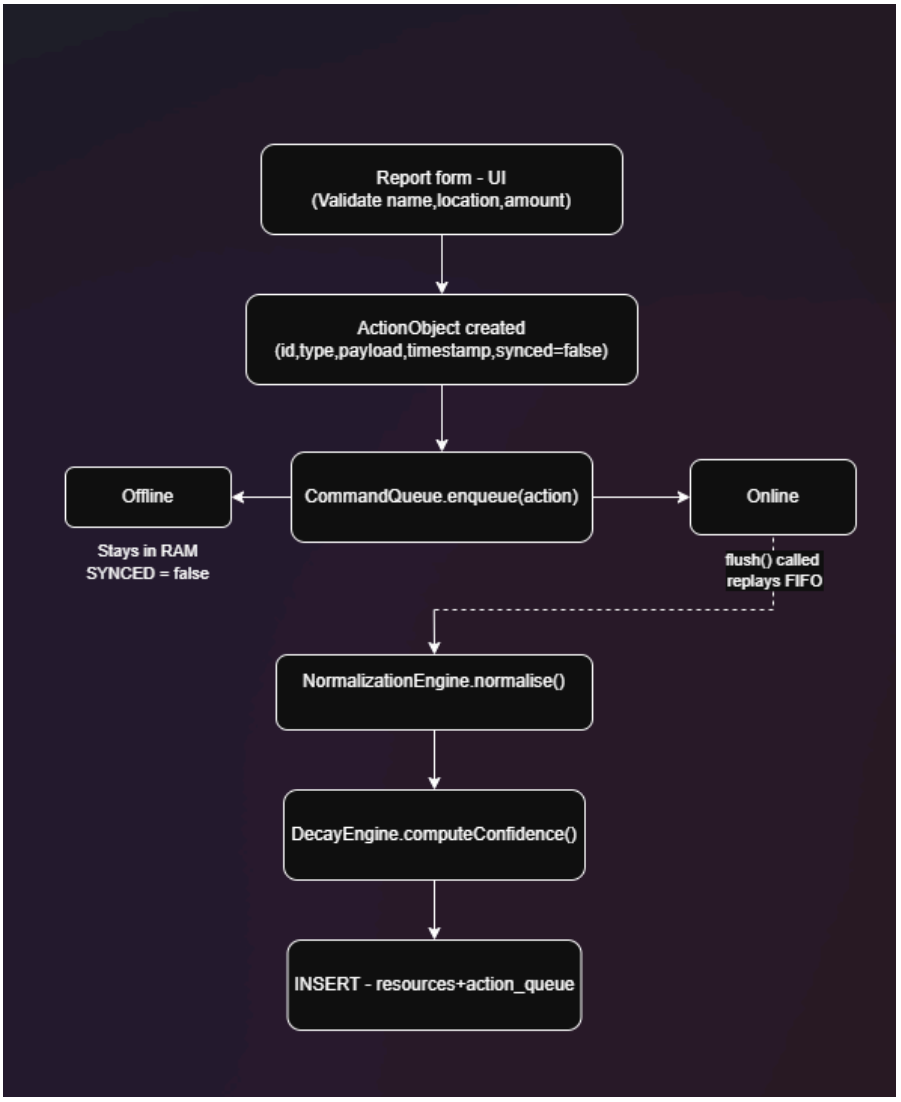
DATA LAYER – MYSQL 8

This layer stores all system data.

Tables include:

- Resources
- Action Queue
- Mesh Nodes

JDBC persistence is used for communication with the database.



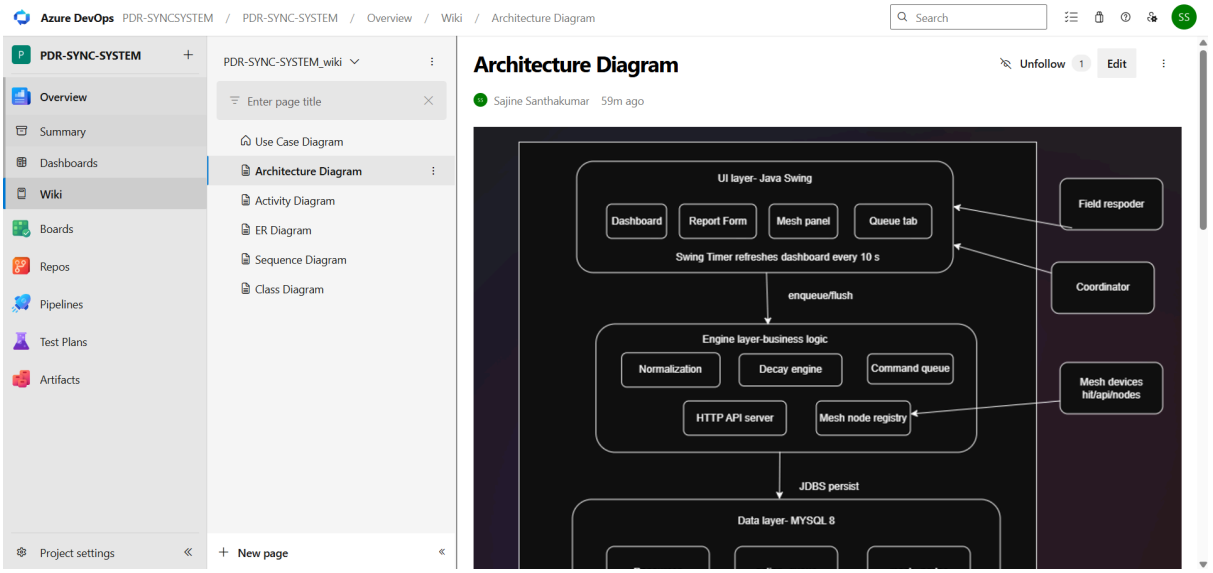
WORKFLOW EXPLANATION

1. User enters disaster report through the UI
2. Input validation is performed
3. An Action Object is created containing:
    - ID
    - Type
    - Payload
    - Timestamp
    - Sync Status
4. The Command Queue stores actions locally
5. If OFFLINE:
    - Data remains in RAM/storage
    - Sync status remains FALSE
6. If ONLINE:
    - Queue flush operation begins

    - Actions replay in FIFO order
7. The Normalization Engine standardizes units
8. The Decay Engine computes confidence score
9. Processed data is inserted into:
    - Resources table
    - Action Queue table

This architecture ensures:

- Offline-first functionality
- Reliable synchronization
- Scalable disaster coordination



RESULT:

Thus, the ARCHITECTURE DIAGRAM for the system “PRECISION DISASTER RECOVERY (PDR-SYNC)” was successfully designed, representing system layers, modules, synchronization flow, and communication architecture.